2943. Maximize Area of Square Hole in Grid

Solved

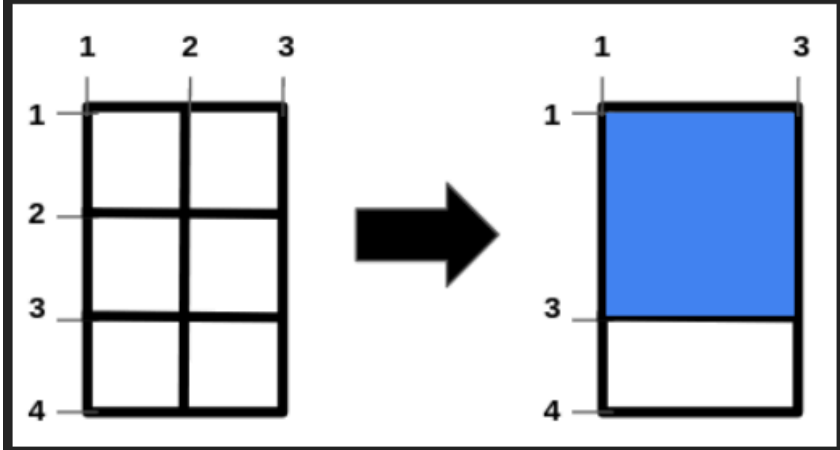
Medium | Topics | Companies | Hint

You are given the two integers, `n` and `m` and two integer arrays, `hBars` and `vBars`. The grid has `n + 2` horizontal and `m + 2` vertical bars, creating 1 x 1 unit cells. The bars are indexed starting from `1`.

You can **remove** some of the bars in `hBars` from horizontal bars and some of the bars in `vBars` from vertical bars. Note that other bars are fixed and cannot be removed.

Return an integer denoting the **maximum area** of a *square-shaped* hole in the grid, after removing some bars (possibly none).

Example 1:



Input: `n = 2, m = 1, hBars = [2,3], vBars = [2]`

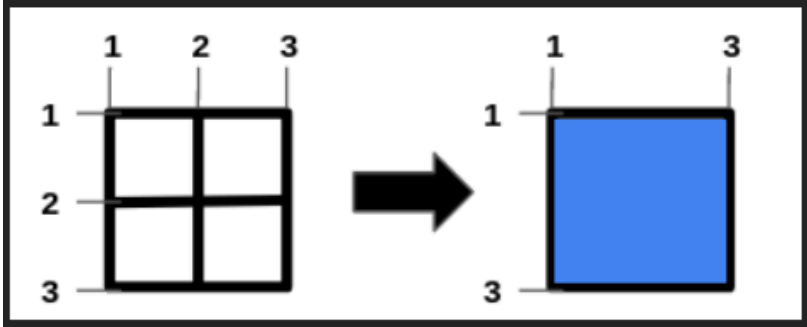
Output: 4

Explanation:

The left image shows the initial grid formed by the bars. The horizontal bars are `[1,2,3,4]`, and the vertical bars are `[1,2,3]`.

One way to get the maximum square-shaped hole is by removing horizontal bar 2 and vertical bar 2.

Example 2:



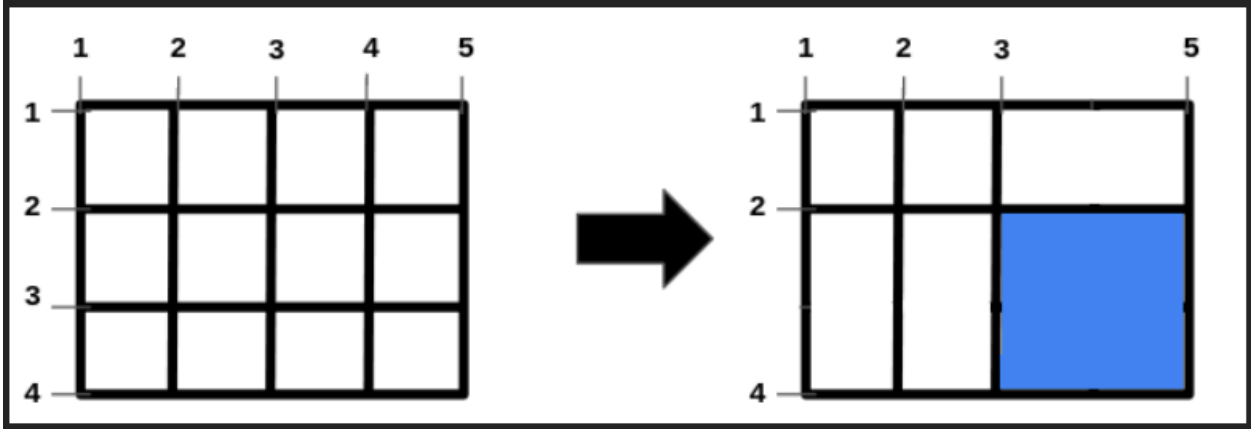
Input: `n = 1, m = 1, hBars = [2], vBars = [2]`

Output: 4

Explanation:

To get the maximum square-shaped hole, we remove horizontal bar 2 and vertical bar 2.

Example 3:

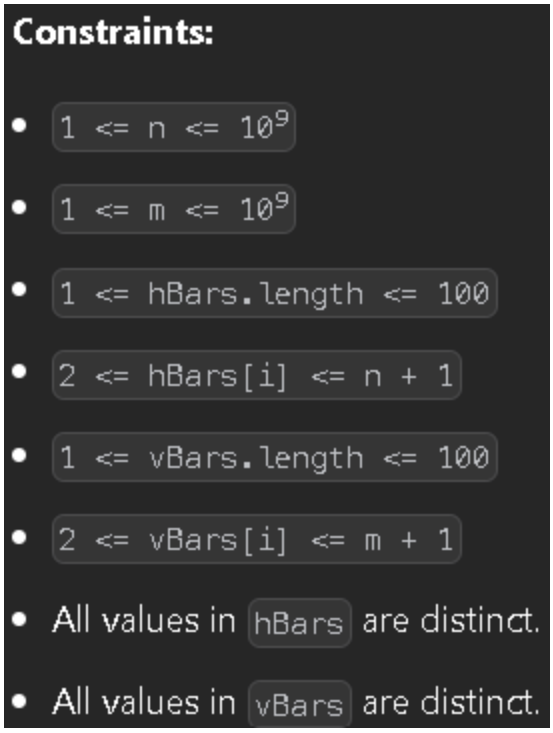


Input: `n = 2, m = 3, hBars = [2,3], vBars = [2,4]`

Output: 4

Explanation:

One way to get the maximum square-shaped hole is by removing horizontal bar 3, and vertical bar 4.



Python:

```
class Solution:
    def maximizeSquareHoleArea(self, n: int, m: int, hBars: list[int], vBars: list[int]) -> int:
        def maxSpan(bars: list[int]) -> int:
            bars.sort()
            res = 1
            streak = 1
            for i in range(1, len(bars)):
                if bars[i] == bars[i - 1] + 1:
                    streak += 1
                    continue
                res = max(res, streak + 1)
                streak = 1
            return max(res, streak + 1)

        return min(maxSpan(hBars), maxSpan(vBars)) ** 2
```

JavaScript:

```
const maximizeSquareHoleArea = (n, m, hBars, vBars) => {
    hBars.sort((a, b) => a - b);
    vBars.sort((a, b) => a - b);

    const maxSpan = bars => {
        let res = 1;
```

```
        let streak = 1;

        for (let i = 1; i < bars.length; i++) {
            if (bars[i] === bars[i - 1] + 1) {
                streak++;
                continue;
            }
            res = Math.max(res, ++streak);
            streak = 1;
        }

        return Math.max(res, ++streak);
    };

    return Math.min(maxSpan(hBars), maxSpan(vBars)) ** 2;
};
```

Java:

```
class Solution {
    public int maximizeSquareHoleArea(int n, int m, int[] hBars, int[] vBars) {
        Arrays.sort(hBars);
        Arrays.sort(vBars);
        return (int)Math.pow(Math.min(maxSpan(hBars), maxSpan(vBars)), 2);
    }

    private int maxSpan(int[] bars) {
        int res = 1;
        int streak = 1;
        for (int i = 1; i < bars.length; i++) {
            if (bars[i] == bars[i - 1] + 1) {
                streak++;
                continue;
            }
            res = Math.max(res, ++streak);
            streak = 1;
        }
        return Math.max(res, ++streak);
    }
}
```